

Environment Enumeration

Enumeration is the key to privilege escalation. Several helper scripts (such as [LinPEAS](#) and [LinEnum](#)) exist to assist with enumeration. Still, it is also important to understand what pieces of information to look for and to be able to perform your enumeration manually. When you gain initial shell access to the host, it is important to check several key details.

OS Version: Knowing the distribution (Ubuntu, Debian, FreeBSD, Fedora, SUSE, Red Hat, CentOS, etc.) will give you an idea of the types of tools that may be available. This would also identify the operating system version, for which there may be public exploits available.

Kernel Version: As with the OS version, there may be public exploits that target a vulnerability in a specific kernel version. Kernel exploits can cause system instability or even a complete crash. Be careful running these against any production system, and make sure you fully understand the exploit and possible ramifications before running one.

Running Services: Knowing what services are running on the host is important, especially those running as root. A misconfigured or vulnerable service running as root can be an easy win for privilege escalation. Flaws have been discovered in many common services such as Nagios, Exim, Samba, ProFTPD, etc. Public exploit PoCs exist for many of them, such as CVE-2016-9566, a local privilege escalation flaw in Nagios Core < 4.2.4.

Gaining Situational Awareness

Let's say we have just gained access to a Linux host by exploiting an unrestricted file upload vulnerability during an External Penetration Test. After establishing our reverse shell (and ideally some sort of persistence), we should start by gathering some basics about the system we are working with.

First, we'll answer the fundamental question: What operating system are we dealing with? If we landed on a CentOS host or Red Hat Enterprise Linux host, our enumeration would likely be slightly different than if we landed on a Debian-based host such as Ubuntu. If we land on a host such as FreeBSD, Solaris, or something more obscure such as the HP proprietary OS HP-UX or the IBM OS AIX, the commands we would work with will likely be different. Though the commands may be different, and we may even need to look up a command reference in some instances, the principles are the same. For our purposes, we'll begin with an Ubuntu target to cover general tactics and techniques. Once we learn the basics and combine them with a new way of thinking and the stages of the Penetration Testing Process, it shouldn't matter what type of Linux system we land on because we'll have a thorough and repeatable process.

There are many cheat sheets out there to help with enumerating Linux systems and some bits of information we are interested in will have two or more ways to obtain it. In this module we'll cover one methodology that can likely be used for the majority of Linux systems that we encounter in the wild. That being said, make sure you understand what the commands are doing and how to tweak them or find the information you need a different way if a particular command doesn't work. Challenge yourself during this module to try things various ways to practice your methodology and what works best for you. Anyone can re-type commands from a cheat sheet but a deep understanding of what you are looking for and how to obtain it will help us be successful in any environment.

Typically we'll want to run a few basic commands to orient ourselves:

- **whoami** - what user are we running as
- **id** - what groups does our user belong to?
- **hostname** - what is the server named. can we gather anything from the naming convention?
- **ifconfig** or **ip -a** - what subnet did we land in, does the host have additional NICs in other subnets?

- `sudo -l` - can our user run anything with sudo (as another user as root) without needing a password?

This can sometimes be the easiest win and we can do something like `sudo su` and drop right into a root shell.

Including screenshots of the above information can be helpful in a client report to provide evidence of a successful Remote Code Execution (RCE) and to clearly identify the affected system. Now let's get into our more detailed, step-by-step, enumeration.

We'll start out by checking out what operating system and version we are dealing with.

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ cat /etc/os-release NAME="Ubuntu" VERSION="20.04.4 LTS (Focal Fossa)" ID=ubuntu ID_LIKE=debian PRETTY_NAME="Ubuntu 20.04.4 LTS" VERSION_ID="20.04" HOME_URL="https://www.ubuntu.com/" SUPPORT_URL="https://help.ubuntu.com/" BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/" PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy" VERSION_CODENAME=focal UBUNTU_CODENAME=focal</pre>

We can see that the target is running **Ubuntu 20.04.4 LTS ("Focal Fossa")**. For whatever version we encounter it's important to see if we're dealing with something out-of-date or maintained. Ubuntu publishes its [release cycle](#) and from this we can see that "Focal Fossa" does not reach end of life until April 2030. From this information we can assume that we will not encounter a well-known Kernel vulnerability because the customer has been keeping their internet-facing asset patched but we'll still look regardless.

Next we'll want to check out our current user's PATH, which is where the Linux system looks every time a command is executed for any executables to match the name of what we type, i.e., `id` which on this system is located at `/usr/bin/id`. As we'll see later in this module, if the PATH variable for a target user is misconfigured we may be able to leverage it to escalate privileges. For now we'll note it down and add it to our notetaking tool of choice.

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ echo \$PATH /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin</pre>

We can also check out all environment variables that are set for our current user, we may get lucky and find something sensitive in there such as a password. We'll note this down and move on.

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ env SHELL=/bin/bash PWD=/home/htb-student LOGNAME=htb-student XDG_SESSION_TYPE=ttty MOTD_SHOWN=pam HOME=/home/htb-student LANG=en_US.UTF-8 <SNIP></pre>

Next let's note down the Kernel version. We can do some searches to see if the target is running a vulnerable Kernel (which we'll get to take advantage of later on in the module) which has some known public exploit PoC. We can do this a few ways, another way would be `cat /proc/version` but we'll use the `uname -a` command.

Environment Enumeration
<pre>dbcyph0n@htb[/htb]\$ uname -a</pre>
<pre>Linux nixlpe02 5.4.0-122-generic #138-Ubuntu SMP Wed Jun 22 15:00:31 UTC 2022 x86_64 x86_64 x86_64 GNU/Linux</pre>

We can next gather some additional information about the host itself such as the CPU type/version:

Environment Enumeration
<pre>dbcyph0n@htb[/htb]\$ lscpu</pre>
<pre>Architecture: x86_64 CPU op-mode(s): 32-bit, 64-bit Byte Order: Little Endian Address sizes: 43 bits physical, 48 bits virtual CPU(s): 2 On-line CPU(s) list: 0,1 Thread(s) per core: 1 Core(s) per socket: 2 Socket(s): 1 NUMA node(s): 1 Vendor ID: AuthenticAMD CPU family: 23 Model: 49 Model name: AMD EPYC 7302P 16-Core Processor Stepping: 0 CPU MHz: 2994.375 BogoMIPS: 5988.75 Hypervisor vendor: VMware</pre>
<pre><SNIP></pre>

What login shells exist on the server? Note these down and highlight that both Tmux and Screen are available to us.

Environment Enumeration
<pre>dbcyph0n@htb[/htb]\$ cat /etc/shells</pre>
<pre># /etc/shells: valid login shells /bin/sh /bin/bash /usr/bin/bash /bin/rbash /usr/bin/rbash /bin/dash /usr/bin/dash /usr/bin/tmux /usr/bin/screen</pre>

We should also check to see if any defenses are in place and we can enumerate any information about them. Some things to look for include:

- Exec Shield
- iptables
- AppArmor

- SELinux
- Fail2ban
- Snort
- Uncomplicated Firewall (ufw)

Often we will not have the privileges to enumerate the configurations of these protections but knowing what, if any, are in place, can help us not to waste time on certain tasks.

Next we can take a look at the drives and any shares on the system. First, we can use the `lsblk` command to enumerate information about block devices on the system (hard disks, USB drives, optical drives, etc.). If we discover and can mount an additional drive or unmounted file system, we may find sensitive files, passwords, or backups that can be leveraged to escalate privileges.

```

Environment Enumeration

dbcypth0n@htb[/htb]$ lsblk

NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
loop0                               7:0      0   55M  1 loop /snap/core18/1705
loop1                               7:1      0   69M  1 loop /snap/lxd/14804
loop2                               7:2      0   47M  1 loop /snap/snapd/16292
loop3                               7:3      0  103M  1 loop /snap/lxd/23339
loop4                               7:4      0   62M  1 loop /snap/core20/1587
loop5                               7:5      0  55.6M  1 loop /snap/core18/2538
sda                                 8:0      0   20G  0 disk
├─sda1                             8:1      0    1M  0 part
├─sda2                             8:2      0    1G  0 part /boot
├─sda3                             8:3      0   19G  0 part
└─ubuntu--vg-ubuntu--lv 253:0    0   18G  0 lvm /
sr0                                 11:0     1   908M  0 rom

```

The command `lpstat` can be used to find information about any printers attached to the system. If there are active or queued print jobs can we gain access to some sort of sensitive information?

We should also checked for mounted drives and unmounted drives. Can we mount an umounted drive and gain access to sensitive data? Can we find any types of credentials in `fstab` for mounted drives by grepping for common words such as password, username, credential, etc in `/etc/fstab`?

```

Environment Enumeration

dbcypth0n@htb[/htb]$ cat /etc/fstab

# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options>          <dump> <pass>
# / was on /dev/ubuntu-vg/ubuntu-lv during curtin installation
/dev/disk/by-id/dm-uuid-LVM-BdLsBLE4CvzJUgtkugkof4S0dZG7gWR8HCN01RdLWoXV0ba2tYUMzHfFQAP9ajul / ext4 defaults 0
# /boot was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/20b1770d-a233-4780-900e-7c99bc974346 /boot ext4 defaults 0 0

```

Check out the routing table by typing `route` or `netstat -rn`. Here we can see what other networks are available via which interface.

```

Environment Enumeration

dbcypth0n@htb[/htb]$ route

Kernel IP routing table

```

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
default	_gateway	0.0.0.0	UG	0	0	0	ens192
10.129.0.0	0.0.0.0	255.255.0.0	U	0	0	0	ens192

In a domain environment we'll definitely want to check `/etc/resolv.conf` if the host is configured to use internal DNS we may be able to use this as a starting point to query the Active Directory environment.

We'll also want to check the arp table to see what other hosts the target has been communicating with.

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ arp -a</pre> <pre>_gateway (10.129.0.1) at 00:50:56:b9:b9:fc [ether] on ens192</pre>

The environment enumeration also includes knowledge about the users that exist on the target system. This is because individual users are often configured during the installation of applications and services to limit the service's privileges. The reason for this is to maintain the security of the system itself. Because if a service is running with the highest privileges (`root`) and this is brought under control by an attacker, the attacker automatically has the highest rights over the entire system. All users on the system are stored in the `/etc/passwd` file. The format gives us some information, such as:

1. Username
2. Password
3. User ID (UID)
4. Group ID (GID)
5. User ID info
6. Home directory
7. Shell

Existing Users

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ cat /etc/passwd</pre> <pre>root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin tcpdump:x:108:115:./nonexistent:/usr/sbin/nologin mrb3n:x:1000:1000:mrb3n:/home/mrb3n:/bin/bash bjones:x:1001:1001:./home/bjones:/bin/sh administrator.ilfreight:x:1002:1002:./home/administrator.ilfreight:/bin/sh backupsvc:x:1003:1003:./home/backupsvc:/bin/sh cliff.moore:x:1004:1004:./home/cliff.moore:/bin/bash logger:x:1005:1005:./home/logger:/bin/sh shared:x:1006:1006:./home/shared:/bin/sh stacey.jenkins:x:1007:1007:./home/stacey.jenkins:/bin/bash</pre>

```
htb-student:x:1008:1008:~/home/htb-student:/bin/bash
<SNIP>
```

Occasionally, we will see password hashes directly in the `/etc/passwd` file. This file is readable by all users, and as with hashes in the `/etc/shadow` file, these can be subjected to an offline password cracking attack. This configuration, while not common, can sometimes be seen on embedded devices and routers.

Environment Enumeration

```
dbcypth0n@htb[/htb]$ cat /etc/passwd | cut -f1 -d:

root
daemon
bin
sys

...SNIP...

mrb3n
lxd
bjones
administrator.ilfreight
backupsvc
cliff.moore
logger
shared
stacey.jenkins
htb-student
```

With Linux, several different hash algorithms can be used to make the passwords unrecognizable. Identifying them from the first hash blocks can help us to use and work with them later if needed. Here is a list of the most used ones:

Algorithm	Hash
Salted MD5	\$1\$...
SHA-256	\$5\$...
SHA-512	\$6\$...
BCrypt	\$2a\$...
Scrypt	\$7\$...
Argon2	\$argon2i\$...

We'll also want to check which users have login shells. Once we see what shells are on the system, we can check each version for vulnerabilities. Because outdated versions, such as Bash version 4.1, are vulnerable to a `shellshock` exploit.

Environment Enumeration

```
dbcypth0n@htb[/htb]$ grep "*sh$" /etc/passwd

root:x:0:0:root:/root:/bin/bash
mrb3n:x:1000:1000:mrb3n:/home/mrb3n:/bin/bash
bjones:x:1001:1001:/home/bjones:/bin/sh
administrator.ilfreight:x:1002:1002:/home/administrator.ilfreight:/bin/sh
backupsvc:x:1003:1003:/home/backupsvc:/bin/sh
cliff.moore:x:1004:1004:/home/cliff.moore:/bin/bash
logger:x:1005:1005:/home/logger:/bin/sh
shared:x:1006:1006:/home/shared:/bin/sh
stacey.jenkins:x:1007:1007:/home/stacey.jenkins:/bin/bash
htb-student:x:1008:1008:/home/htb-student:/bin/bash
```

Each user in Linux systems is assigned to a specific group or groups and thus receives special privileges. For example, if we have a folder named `dev` only for developers, a user must be assigned to the appropriate group to access that folder. The information about the available groups can be found in the `/etc/group` file, which shows us both the group name and the assigned user names.

Existing Groups

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ cat /etc/group root:x:0: daemon:x:1: bin:x:2: sys:x:3: adm:x:4:syslog,htb-student tty:x:5:syslog disk:x:6: lp:x:7: mail:x:8: news:x:9: uucp:x:10: man:x:12: proxy:x:13: kmem:x:15: dialout:x:20: fax:x:21: voice:x:22: cdrom:x:24:htb-student floppy:x:25: tape:x:26: sudo:x:27:mr3n,htb-student audio:x:29:pulse dip:x:30:htb-student www-data:x:33: ...SNIP...</pre>

The `/etc/group` file lists all of the groups on the system. We can then use the `getent` command to list members of any interesting groups.

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ getent group sudo sudo:x:27:mr3n</pre>

We can also check out which users have a folder under the `/home` directory. We'll want to enumerate each of these to see if any of the system users are storing any sensitive data, files containing passwords. We should check to see if files such as the `.bash_history` file are readable and contain any interesting commands and look for configuration files. It is not uncommon to find files containing credentials that can be leveraged to access other systems or even gain entry into the Active Directory environment. Its also important to check for SSH keys for all users, as these could be used to achieve persistence on the system, potentially to escalate privileges, or to assist with pivoting and port forwarding further into the internal network. At the minimum, check the ARP cache to see what other hosts are being accessed and cross-reference these against any useable SSH private keys.

Environment Enumeration
<pre>dbcypth0n@htb[/htb]\$ ls /home administrator.ilfreight bjones htb-student mr3n stacey.jenkins</pre>


```
backupsvc      cliff.moore  logger      shared
```

Finally, we can search for any "low hanging fruit" such as config files, and other files that may contain sensitive information.

Configuration files can hold a wealth of information. It is worth searching through all files that end in extensions such as .conf and .config, for usernames, passwords, and other secrets.

If we've gathered any passwords we should try them at this time for all users present on the system. Password re-use is common so we might get lucky!

In Linux, there are many different places where such files can be stored, including mounted file systems. A mounted file system is a file system that is attached to a particular directory on the system and accessed through that directory. Many file systems, such as ext4, NTFS, and FAT32, can be mounted. Each type of file system has its own benefits and drawbacks. For example, some file systems can only be read by the operating system, while others can be read and written by the user. File systems that can be read and written to by the user are called read/write file systems. Mounting a file system allows the user to access the files and folders stored on that file system. In order to mount a file system, the user must have root privileges. Once a file system is mounted, it can be unmounted by the user with root privileges. We may have access to such file systems and may find sensitive information, documentation, or applications there.

Mounted File Systems

Environment Enumeration					
dbcypth0n@htb[/htb]\$ df -h					
Filesystem	Size	Used	Avail	Use%	Mounted on
udev	1,9G	0	1,9G	0%	/dev
tmpfs	389M	1,8M	388M	1%	/run
/dev/sda5	20G	7,9G	11G	44%	/
tmpfs	1,9G	0	1,9G	0%	/dev/shm
tmpfs	5,0M	4,0K	5,0M	1%	/run/lock
tmpfs	1,9G	0	1,9G	0%	/sys/fs/cgroup
/dev/loop0	128K	128K	0	100%	/snap/bare/5
/dev/loop1	62M	62M	0	100%	/snap/core20/1611
/dev/loop2	92M	92M	0	100%	/snap/gtk-common-themes/1535
/dev/loop4	55M	55M	0	100%	/snap/snap-store/558
/dev/loop3	347M	347M	0	100%	/snap/gnome-3-38-2004/115
/dev/loop5	47M	47M	0	100%	/snap/snapd/16292
/dev/sda1	511M	4,0K	511M	1%	/boot/efi
tmpfs	389M	24K	389M	1%	/run/user/1000
/dev/sr0	3,6G	3,6G	0	100%	/media/htb-student/Ubuntu 20.04.5 LTS amd64
/dev/loop6	50M	50M	0	100%	/snap/snapd/17576
/dev/loop7	64M	64M	0	100%	/snap/core20/1695
/dev/loop8	46M	46M	0	100%	/snap/snap-store/599
/dev/loop9	347M	347M	0	100%	/snap/gnome-3-38-2004/119

When a file system is unmounted, it is no longer accessible by the system. This can be done for various reasons, such as when a disk is removed, or a file system is no longer needed. Another reason may be that files, scripts, documents, and other important information must not be mounted and viewed by a standard user. Therefore, if we can extend our privileges to the `root` user, we could mount and read these file systems ourselves. Unmounted file systems can be viewed as follows:

Unmounted File Systems

Environment Enumeration					
dbcypth0n@htb[/htb]\$ cat /etc/fstab grep -v "#" column -t					
UUID=5bdf16727-fcdf-4205-906c-0620aa4a058f	/	ext4	errors=remount-ro	0	1
UUID=BE56-AAE0	/boot/efi	vfat	umask=0077	0	1

/swapfile

none

swap sw

0 0

Many folders and files are kept hidden on a Linux system so they are not obvious, and accidental editing is prevented. Why such files and folders are kept hidden, there are many more reasons than those mentioned so far. Nevertheless, we need to be able to locate all hidden files and folders because they can often contain sensitive information, even if we have read-only permissions.

All Hidden Files

Environment Enumeration

```
dbcyph0n@htb[/htb]$ find / -type f -name ".*" -exec ls -l {} \; 2>/dev/null | grep htb-student
```

```
-rw-r--r-- 1 htb-student htb-student 3771 Nov 27 11:16 /home/htb-student/.bashrc
-rw-rw-r-- 1 htb-student htb-student 180 Nov 27 11:36 /home/htb-student/.wget-hsts
-rw----- 1 htb-student htb-student 387 Nov 27 14:02 /home/htb-student/.bash_history
-rw-r--r-- 1 htb-student htb-student 807 Nov 27 11:16 /home/htb-student/.profile
-rw-r--r-- 1 htb-student htb-student 0 Nov 27 11:31 /home/htb-student/.sudo_as_admin_successful
-rw-r--r-- 1 htb-student htb-student 220 Nov 27 11:16 /home/htb-student/.bash_logout
-rw-rw-r-- 1 htb-student htb-student 162 Nov 28 13:26 /home/htb-student/.notes
```

All Hidden Directories

Environment Enumeration

```
dbcyph0n@htb[/htb]$ find / -type d -name ".*" -ls 2>/dev/null
```

```

684822      4 drwx----- 3 htb-student htb-student 4096 Nov 28 12:32 /home/htb-student/.gnupg
790793      4 drwx----- 2 htb-student htb-student 4096 Oct 27 11:31 /home/htb-student/.ssh
684804      4 drwx----- 10 htb-student htb-student 4096 Oct 27 11:30 /home/htb-student/.cache
790827      4 drwxrwxr-x 8 htb-student htb-student 4096 Oct 27 11:32 /home/htb-student/CVE-2021-3156/
684796      4 drwx----- 10 htb-student htb-student 4096 Oct 27 11:30 /home/htb-student/.config
655426      4 drwxr-xr-x 3 htb-student htb-student 4096 Oct 27 11:19 /home/htb-student/.local
524808      4 drwxr-xr-x 7 gdm          gdm          4096 Oct 27 11:19 /var/lib/gdm3/.cache
544027      4 drwxr-xr-x 7 gdm          gdm          4096 Oct 27 11:19 /var/lib/gdm3/.config
544028      4 drwxr-xr-x 3 gdm          gdm          4096 Aug 31 08:54 /var/lib/gdm3/.local
524938      4 drwx----- 2 colord       colord       4096 Oct 27 11:19 /var/lib/colord/.cache
  1408      2 dr-xr-xr-x 1 htb-student htb-student 2048 Aug 31 09:17 /media/htb-student/Ubuntu\ 20.04
280101      4 drwxrwxrwt 2 root          root          4096 Nov 28 12:31 /tmp/.font-unix
262364      4 drwxrwxrwt 2 root          root          4096 Nov 28 12:32 /tmp/.ICE-unix
262362      4 drwxrwxrwt 2 root          root          4096 Nov 28 12:32 /tmp/.X11-unix
280103      4 drwxrwxrwt 2 root          root          4096 Nov 28 12:31 /tmp/.Test-unix
262830      4 drwxrwxrwt 2 root          root          4096 Nov 28 12:31 /tmp/.XIM-unix
661820      4 drwxr-xr-x 5 root          root          4096 Aug 31 08:55 /usr/lib/modules/5.15.0-46-generic
666709      4 drwxr-xr-x 5 root          root          4096 Oct 27 11:18 /usr/lib/modules/5.15.0-52-generic
657527      4 drwxr-xr-x 170 root         root          4096 Aug 31 08:55 /usr/lib/debug/.build-id
```

In addition, three default folders are intended for temporary files. These folders are visible to all users and can be read. In addition, temporary logs or script output can be found there. Both `/tmp` and `/var/tmp` are used to store data temporarily. However, the key difference is how long the data is stored in these file systems. The data retention time for `/var/tmp` is much longer than that of the `/tmp` directory. By default, all files and data stored in `/var/tmp` are retained for up to 30 days. In `/tmp`, on the other hand, the data is automatically deleted after ten days.

In addition, all temporary files stored in the `/tmp` directory are deleted immediately when the system is restarted. Therefore, the `/var/tmp` directory is used by programs to store data that must be kept between reboots temporarily.

Temporary Files

Environment Enumeration

```
dbcyph0n@htb[/htb]$ ls -l /tmp /var/tmp /dev/shm
```

```

/dev/shm:
total 0

/tmp:
total 52
-rw----- 1 htb-student htb-student 0 Nov 28 12:32 config-err-v8LfEU
drwx----- 3 root root 4096 Nov 28 12:37 snap.snap-store
drwx----- 2 htb-student htb-student 4096 Nov 28 12:32 ssh-0KLLKjlc98xh
<SNIP>
drwx----- 2 htb-student htb-student 4096 Nov 28 12:37 tracker-extract-files.1000
drwx----- 2 gdm gdm 4096 Nov 28 12:31 tracker-extract-files.125

/var/tmp:
total 28
drwx----- 3 root root 4096 Nov 28 12:31 systemd-private-7b455e62ec09484b87eff41023c4ca53-colord.service-RrPcy
drwx----- 3 root root 4096 Nov 28 12:31 systemd-private-7b455e62ec09484b87eff41023c4ca53-ModemManager.service
...SNIP...

```

Moving On

We've gotten an initial lay of the land and (hopefully) some sensitive or useful data points that can help us on our way to escalating privileges or even moving laterally in the internal network. Next we'll turn our focus to permissions and check to see what directories, scripts, binaries, etc we can read and write to with our current user privileges.

Though we are focusing on manual enumeration in this module, its worth running the [linPEAS](#) script at this point in a real-world assessment so we have as much data to dig through as possible. Oftentimes we can find an easy win but having this output handy can sometimes uncover nuanced issues that our manual enumeration missed. We should, though, practice our manual enumeration as much as possible and create (and continue to add to) our own cheat sheet of key commands (and alternatives for different Linux operating systems). We'll start to develop our own style, command preference, and even see some areas that we can begin to script out ourselves. Tools are great and have their place but where many fall short is being able to perform a given task when a tool fails or we cannot get it onto the system.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

EU Academy 3

 Recommended Medium Load ▼

PROTOCOL

 UDP 1337  TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

34ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): [Click here to spawn the target system!](#)

SSH to with user "htb-student" and password "HTB_@cademy_stdnt!"

+0 Enumerate the Linux environment and look for interesting files that might contain sensitive data. Submit the flag as the answer.

Submit your answer here...

+10 Streak pts

Submit

Show Solution

Previous

Next

Cheat Sheet

? Go to Questions

Table of Contents

Introduction

Introduction to Linux Privilege Escalation



Information Gathering

 [Environment Enumeration](#)

 [Linux Services & Internals Enumeration](#)

 [Credential Hunting](#)

Environment-based Privilege Escalation

 [Path Abuse](#)

[Wildcard Abuse](#)

 [Escaping Restricted Shells](#)

Permissions-based Privilege Escalation

 [Special Permissions](#)

 [Sudo Rights Abuse](#)

 [Privileged Groups](#)

 [Capabilities](#)

Service-based Privilege Escalation

 [Vulnerable Services](#)

 [Cron Job Abuse](#)

 [LXD](#)

 [Docker](#)

[Kubernetes](#)

 [Logrotate](#)

 [Miscellaneous Techniques](#)

Linux Internals-based Privilege Escalation

 [Kernel Exploits](#)

 [Shared Libraries](#)

 [Shared Object Hijacking](#)

 [Python Library Hijacking](#)

Recent 0-Days

 [Sudo](#)

 [Polkit](#)

 [Dirty Pipe](#)

[Netfilter](#)

Hardening Considerations

[Linux Hardening](#)

Skills Assessment

 [Linux Local Privilege Escalation - Skills Assessment](#)

My Workstation

OFFLINE

▶ Start Instance

∞ / 1 spawns left